

Automation On A Schedule

Claude Code Advanced, episode 4

Three places the work can run. The question that picks one is whether your machine will be on.

You want the deploy checked every five minutes, so you type `/loop` and move on. Where is that actually running, and what happens to it when you close the terminal? Most people find out at eleven at night.

This sheet is the three-way choice, the three shapes `/loop` takes, every limit that decides whether it is the right tool, and one piece of security design in routines that deserves more attention than it gets.

Claude Code changes with every release. Every figure, default and feature name on this sheet was checked on the date below and is stated as of that date. Check the current documentation before you rely on any of it.

In scope

- Cloud routines, Desktop tasks and `/loop`, and where each one runs
- The three shapes of `/loop`, including the one with no prompt
- `loop.md`, and why edits apply mid-loop
- What a scheduled fire will and will not execute
- Session scope, expiry, catch-up and jitter
- Routines: triggers, autonomy, and the connector default
- The untrusted-data wrapper on the fire endpoint

Out of scope

- GitHub Actions, which is a different page and a different product
- Cron syntax beyond what the tool accepts
- Any claim that automation reduces the work. It moves it
- Long-running agents, which is the subagents episode

Read the first table before anything else. Every limit further down is a consequence of which row you picked.

01 Where the work runs, checked September 2026

Cloud routine	Anthropic cloud. Machine off is fine. No local files
Desktop task	Your machine. No open session needed. Local files yes
/loop	Your machine AND an open session. Local files yes
Minimum interval, cloud	1 hour
Minimum interval, Desktop and /loop	1 minute
Scheduled tasks per session	50
Recurring task lifetime	7 days, then one final fire and it deletes itself
Recurring jitter	up to 30 minutes late, or half the interval if sub-hourly
One-shot jitter on the hour	up to 90 seconds EARLY
loop.md size	truncated past 25,000 bytes

The middle column decides everything. Two of the three need a machine of yours switched on, and one of those also needs a session open in front of you.

03 The three-way choice, which is the whole subject

A CLOUD ROUTINE runs on Anthropic-managed infrastructure. Your machine can be off. It works from a fresh clone, so it has no access to your local files.

A DESKTOP TASK runs on your machine and does not need a session open, and it can reach your local files and tools.

/LOOP runs on your machine and needs the session open, because tasks fire only while Claude Code is running and idle.

The documentation's own rule is the right one: cloud tasks for work that should run reliably without your machine, Desktop tasks when you need local files and tools, /loop for quick polling during a session.

Use cloud tasks for work that should run reliably without your machine. Use Desktop tasks when you need access to local files and tools. Use /loop for quick polling during a session.

Claude Code docs, Scheduled tasks, checked September 2026

Minimum interval is 1 hour for cloud, 1 minute for Desktop tasks and /loop.

Claude Code docs, Scheduled tasks, checked September 2026

04 Three shapes, one command

INTERVAL AND PROMPT, ``/loop 5m check the deploy``: runs on a fixed cron schedule. The interval can lead as a bare token or trail as "every 2 hours". Intervals that do not map to a clean cron step, such as 7m or 90m, are rounded and Claude tells you what it picked.

PROMPT ONLY, ``/loop check the deploy``: Claude chooses the interval itself after each iteration, between one minute and one hour, based on what it just observed. Short waits while a build is finishing, longer ones once the PR goes quiet. THE DELAY AND THE REASON ARE PRINTED each iteration, which is the only feedback a self-paced loop gives you and is worth actually reading.

NEITHER, a bare ``/loop``: the built-in maintenance prompt runs, at a dynamically chosen interval. Add an interval to run it on a fixed schedule instead.

You can also pass a skill: ``/loop 20m /review-pr 1234`` re-runs that skill each iteration.

When you omit the interval, Claude picks a delay between one minute and one hour based on what it observed. The chosen delay and the reason for it are printed at the end of each iteration.

Claude Code docs, Scheduled tasks, checked September 2026

05 The bare /loop is not a no-op

With no prompt, Claude works through a fixed list, in order: continue any unfinished work from the conversation; tend to the current branch's pull request, meaning review comments, failed CI runs and merge conflicts; then run cleanup passes such as bug hunts or simplification when nothing else is pending.

Two limits are written into it, and the second is the reason you can leave it running. Claude does not start new initiatives outside that scope. And irreversible actions such as pushing or deleting only proceed when they CONTINUE something the transcript already authorized.

To replace it with your own, create `loop.md`: `.claude/loop.md` for the project, `~/.claude/loop.md` for the user, project wins. It is plain Markdown written as if you were typing the prompt.

EDITS TAKE EFFECT ON THE NEXT ITERATION, so you can watch a loop work and sharpen its instructions without ever stopping it. Keep it under 25,000 bytes or it is truncated.

Claude does not start new initiatives outside that scope, and irreversible actions such as pushing or deleting only proceed when they continue something the transcript already authorized.

Claude Code docs, Scheduled tasks, checked September 2026

Edits to loop.md take effect on the next iteration, so you can refine the instructions while a loop is running.

Claude Code docs, Scheduled tasks, checked September 2026

06 What a scheduled fire will not execute

A scheduled fire only runs skills that Claude is allowed to invoke on its own. Everything else reaches Claude AS PLAIN TEXT instead of executing, which is a confusing failure to debug if you do not know the shape of it.

That covers built-in commands such as `/permissions`, `/model` or `/clear`; skills marked `disable-model-invocation: true`, including the bundled `/verify` skill; skills withheld by a `SkillOverrides` setting or a Skill deny rule; and MCP prompts.

So a loop that looks like it is running your command every ten minutes, and is actually sending Claude the words, is a real thing that happens.

A scheduled fire only runs skills that Claude is allowed to invoke on its own. Built-in commands, skills marked `disable-model-invocation`, withheld skills and MCP prompts reach Claude as plain text instead.

Claude Code docs, Scheduled tasks, checked September 2026

07 The limits that decide whether this is the right tool

SESSION SCOPED. Tasks fire only while Claude Code is running and idle. Closing the terminal stops them. Backgrounding the session carries /loop tasks over to a background session, which is the escape hatch.

NO CATCH-UP. If a task's time passes while Claude is busy, it fires once when Claude becomes idle, not once per missed interval.

SEVEN-DAY EXPIRY. Recurring tasks expire 7 days after creation: one final fire, then the task deletes itself. That is a feature, not a limitation, because it bounds how long a forgotten loop can run, which is the exact failure the token economy episode is about.

RESUME IS PARTIAL. `--resume` restores tasks scheduled with `CronCreate`, but not expired recurring ones, not one-shots whose time has passed, and NOT a self-paced /loop. Run /loop again to restart that one.

And `Esc` stops a self-paced loop while it is waiting. Tasks you scheduled by asking Claude directly are not affected by `Esc`.

Recurring tasks automatically expire 7 days after creation. The task fires one final time, then deletes itself. This bounds how long a forgotten loop can run.

Claude Code docs, Scheduled tasks, checked September 2026

A self-paced /loop is not restored on resume, so run /loop again to restart it.

Claude Code docs, Scheduled tasks, checked September 2026

08 Fire times are deliberately not exact

To avoid every session hitting the API at the same wall-clock moment, the scheduler adds a deterministic offset.

Recurring tasks fire up to 30 minutes AFTER the scheduled time, or up to half the interval for tasks that run more often than hourly. An hourly job scheduled for :00 may fire anywhere up to :30.

One-shot tasks scheduled for the top or bottom of the hour fire up to 90 seconds EARLY.

The offset is derived from the task ID, so the same task always gets the same offset. Which gives you a genuinely useful trick: if exact timing matters, pick a minute that is not :00 or :30, for example ``3 9 * * *``, and the one-shot jitter does not apply at all.

Recurring tasks fire up to 30 minutes after the scheduled time, or up to half the interval for tasks that run more often than hourly. One-shot tasks scheduled for the top or bottom of the hour fire up to 90 seconds early.

Claude Code docs, Scheduled tasks, checked September 2026

09 Routines, and the blast radius that comes with them

At the time of writing, a research preview on Pro, Max, Team and Enterprise. Three trigger types, combinable on one routine: a schedule, an API call, and GitHub events.

They run autonomously, which means NO PERMISSION PROMPTS during a run. Read that with the blast radius episode in mind: autonomy, plus write access, plus nobody watching.

Connectors are included by default, and Claude can use every tool from an included connector, INCLUDING WRITES, without asking. Remove the ones the routine does not need, and treat the remaining list as the answer to "what can this thing reach at three in the morning".

And a green status means the session started and exited. It does NOT mean the task succeeded. Owners can disable routines organisation-wide.

Routines run autonomously with no permission prompts during a run, and Claude can use every tool from an included connector, including writes, without asking.

Claude Code docs, Routines, checked September 2026

A green status means the session started and exited, not that the task succeeded.

Claude Code docs, Routines, checked September 2026

10 The best piece of design in the product

You can fire a routine by sending text to an API endpoint. The obvious question is what stops somebody using that to send it instructions.

The answer: text sent to the fire endpoint ARRIVES WRAPPED IN A BLOCK THAT LABELS IT UNTRUSTED DATA, and the routine's own prompt must explicitly opt in before it will act on any of it.

The reasoning is stated plainly: anyone holding the bearer token can send that text, so it arrives labelled as data rather than as instructions.

That is prompt injection defence designed into an API rather than bolted on afterwards, and it is the clearest real-world answer to the MCP warning that servers fetching external content can expose you to prompt injection risk. Worth copying in your own systems, whether or not you ever use a routine.

Text sent to the fire endpoint arrives wrapped in a block labelling it untrusted data, and the routine prompt must explicitly opt in to acting on it, because anyone holding the bearer token can send that text.

Claude Code docs, Routines, checked September 2026

Servers that fetch external content can expose you to prompt injection risk.

Claude Code docs, MCP, checked September 2026



Before you schedule anything

1. Does this need to survive my laptop being shut? Then it is not /loop
2. Does it need my local files? Then it is not a cloud routine
3. Is an hour often enough? Cloud cannot go faster
4. Self-paced? Am I reading the delay and the reason it prints?
5. Is the thing I am looping actually a skill Claude may invoke itself?
6. Have I understood that a recurring task dies after seven days?
7. Does exact timing matter? Then not :00 and not :30
8. For a routine: which connectors did I leave in, and what can they write?
9. For a routine: am I treating green as "it ran" rather than "it worked"?
10. Is anything still looping that I have forgotten about? Check /usage

Everything else

Site

<https://code-with-sam-dev.github.io>

YouTube

<https://www.youtube.com/@CodewithSam-Dev>

LinkedIn

<https://www.linkedin.com/company/code-with-sam-dev>

X

<https://x.com/CodeWithSamDev>

TikTok

<https://www.tiktok.com/@codewithsamdev>

Instagram

<https://www.instagram.com/codewithsamdev>

Facebook

<https://www.facebook.com/codewithsamdev>

GitHub

<https://github.com/code-with-sam-dev>

☕ Buy me a coffee

<https://buymeacoffee.com/codewithsam>

Buy me sadza

<https://buymeacoffee.com/codewithsam>

Scheduled tasks

<https://code.claude.com/docs/en/scheduled-tasks>

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Claude and Claude Code are products of Anthropic. This is an independent, unofficial guide produced by Code with Sam. It is not affiliated with, endorsed by, or sponsored by Anthropic, and no Anthropic artwork is reproduced here. Product names appear only to identify the software being discussed. Other names are the property of their respective owners.

Technical claims verified 2026-09-13 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.